- ASP.NET Identity supports roles-based authorization, which enables you to group together users who have the same permissions.
- You can use the current user's claims as the basis for authorization, which represent data about the user and form part of their identity.
- Authorization is built on requirements, handlers, and policies. Each policy is formed of one or more requirements, and each requirement has one or more handlers.
- A requirement handler contains the logic for determining whether the current user is authorized. The handler calls `context.Succeed(requirement)` to mark the requirement as succeeded.
- Only one handler needs to succeed for a requirement to succeed. All requirements must be met for authorization to be granted.
- You can ensure authorization fails by calling the `context.Fail` method within a handler.
- Authorization includes a number of built-in policies, including ones that require one or more roles, one or more claims, an authenticated user, and a specific username.
- You can create your own custom requirements, handlers, and policies. Custom handlers must be registered as services.
- Resource authorization enables you to apply fine-grained control over access to elements of your application model.
- The `OperationAuthorizationRequirement` helper class is intended to be used to help define handlers for authorizing specific operations on resources.
- You can inject the `IAuthorizationService` into your Razor page to apply authorization to the rendering of UI.